Lesson4_R_Datatypes

Paul Lott

2024-05-13

R Week Review

- **Commenting:** Comments start with `#`, the `#` mark can at any position on the line. Everything on the line after the `#` is considered a comment
- **Variables:** Variables can be assigned in three ways. `=`, `<-`, or `->` . Variables start with alphabetic characters and can contain alphanumeric characters, and/or underscores. Variables names are case-sensitive.
  - variable = value :
  - variable <- value
  - value -> variable
- **Functions:** Functions in R are called with the function name followed directly by an opening parenthese (Then comes the arguments, and/or paramenters separated by commas and its completed with a closing parenthese)
  - function name(options/parameters separated by commas)
- **Printing with the `print` function:** If you want to print out a variable value or string, the print() function takes the variable or string
  - `print(x) # This will print the value of x`
  - `print("Hello World")  # Will print Hello World`

Formatted Print

R provides a formatted print function called `sprintf()` that takes a format string followed by values to fill in to the format string. Parameters are separated with commas.

```
apple_cnt = 10
name = "Billy"
sprintf("%s has %s apples.", name, apple_cnt)
```

```
## [1] "Billy has 10 apples."
```

sprintf C-Style Format Characters `sprintf` is a C-style string formatting function that allows you to format numbers to display properly- (ie Significant Digits, or Scientific Notation)

Notation	Description
%s	a string
%d	an integer
%0xd	an integer padded with x leading zeros
%f	decimal notation with six decimals
%.xf	floating point number with x digits after decimal point
%e	compact scientific notation, e in the exponent
%E	compact scientific notation, E in the exponent
%g	compact decimal or scientific notation (with e)
%o	integer in octal form
%x	Integer in hexadecimal form

```r
y = sprintf("Mean: %s", 1.223223)  # As an string
print(y)
## [1] "Mean: 1.223223"
y = sprintf("Mean: %d", 2)  # As an Integer
print(y)
## [1] "Mean: 2"
y = sprintf("Mean: %03d", 2) # As an Integer with 3 Padded Zeros
print(y)
## [1] "Mean: 002"
y = sprintf("Mean: %f", 1.223223) # As a floting point number
print(y)
## [1] "Mean: 1.223223"
y = sprintf("Mean: %.03f", 1.223223)  # As a floating point number with 3 digits after decimal point
print(y)
## [1] "Mean: 1.223"
y = sprintf("Mean: %e", 0.000123213)  # As a floating point in compact scientific 'e' notation
print(y)
## [1] "Mean: 1.232130e-04"
y = sprintf("Mean: %E", 0.000232212) # As a floating point in compact scientific 'E' notation
print(y)
## [1] "Mean: 2.322120E-04"
y = sprintf("Mean: %g", 0.0230123) # Compact decimal or scientific notation with 'e'
print(y)
## [1] "Mean: 0.0230123"
y = sprintf("Mean: %o", 20)
print(y)
## [1] "Mean: 24"
```

We'll see alot more on this in the sections about Character/String Datatypes

---

I. Atomic Datatypes

Datatypes are a fundamental concept in programming. Computer's work on binary switches, so everything in RAM, CPU, HDD is composed of **0,1**

For example, the binary value **01100001** has different meaning depending on the type of data. If it represents an integer, then it has a value of **97**, but if it's a character/string it's an **a.** Programming languages assign a data type to the data, which determines how the data is represented.

Datatypes

R has six basic fundamental datatypes. These are single instances of fundamental computer datatypes stored in memory.

R Datatype	Description
Complex	Set of all the complex numbers (9+2i, where i is the imaginary part)
Raw	Byte encoded
Logical (aka Boolean)	Logical/Boolean either TRUE or FALSE
Numeric	Decimal values are called numeric in R. It is the default R data type for numbers in R. Example: 10.4, 55, 78.121E-2
Integer	Integer data types which are the set of all integers. (Must have suffix `L`) Example: 10L, 1L, 55L
Character (aka String)	Alphabet and Special Characters (ASCII) Example: "Hello", "World"

Helpful Functions to determine datatypes and classes

R has functions that returns specific attributes about the datatype of data.

Table 3: Let's look at some different Datatypes and use the these functions.

Function	Description
class ()	Returns type of container of an object
mode()	Returns how the object is stored in memory
typeof()	Returns type of object that is in the container
is.numeric()	Returns TRUE if type is `numeric`
is.logical()	Returns TRUE if type is `logical`
is.complex()	Returns TRUE if type is `complex`
is.integer()	Returns TRUE if type is `integer`
is.raw()	Returns TRUE if type is `raw`
is.character()	Returns TRUE if type is `character`

A. Complex

```
x <-1i
print(paste("CLASS:", class(x)))
## [1] "CLASS: complex"
print(paste("MODE:", mode(x)))
## [1] "MODE: complex"
print(paste("TYPEOF:", typeof(x)))
## [1] "TYPEOF: complex"
print(paste("IS.NUMERIC:", is.numeric(x)))
## [1] "IS.NUMERIC: FALSE"
print(paste("IS.LOGICAL:", is.logical(x)))
```

```r
## [1] "IS.LOGICAL: FALSE"
print(paste("IS.COMPLEX:", is.complex(x)))
## [1] "IS.COMPLEX: TRUE"
print(paste("IS.INTEGER:", is.integer(x)))
## [1] "IS.INTEGER: FALSE"
print(paste("IS.RAW:", is.raw(x)))
## [1] "IS.RAW: FALSE"
print(paste("IS.CHARACTER:", is.character(x)))
## [1] "IS.CHARACTER: FALSE"
```

B. Raw

```r
x <-raw(1)
print(paste("CLASS:", class(x)))
## [1] "CLASS: raw"
print(paste("MODE:", mode(x)))
## [1] "MODE: raw"
print(paste("TYPEOF:", typeof(x)))
## [1] "TYPEOF: raw"
print(paste("IS.NUMERIC:", is.numeric(x)))
## [1] "IS.NUMERIC: FALSE"
print(paste("IS.LOGICAL:", is.logical(x)))
## [1] "IS.LOGICAL: FALSE"
print(paste("IS.COMPLEX:", is.complex(x)))
## [1] "IS.COMPLEX: FALSE"
print(paste("IS.INTEGER:", is.integer(x)))
## [1] "IS.INTEGER: FALSE"
print(paste("IS.RAW:", is.raw(x)))
## [1] "IS.RAW: TRUE"
print(paste("IS.CHARACTER:", is.character(x)))
## [1] "IS.CHARACTER: FALSE"
```

C. Logical

Boolean/Logical types are binary TRUE or FALSE. Typically, used in conditionals.

Note: Boolean in R are represented as `TRUE` is true and `FALSE` is false. It must be all caps.

```r
TRUE
## [1] TRUE
FALSE
## [1] FALSE

true
## Error: object 'true' not found
false
## Error: object 'false' not found

True
```

```
## Error: object 'True' not found
False
## Error: object 'False' not found
```

Note: If you use non-capital `true` or `false` - R interprets these as variable names and not `TRUE` or `FALSE`

There's one additional **Logical** value. In R, there is a Logical value that is equivalent to `None` - it is `NA`, which stands for **Not Available.**

```
x = NA
print(x)
```

```
## [1] NA
```

```
x <-TRUE
print(paste("CLASS:", class(x)))
## [1] "CLASS: logical"
print(paste("MODE:", mode(x)))
## [1] "MODE: logical"
print(paste("TYPEOF:", typeof(x)))
## [1] "TYPEOF: logical"
print(paste("IS.NUMERIC:", is.numeric(x)))
## [1] "IS.NUMERIC: FALSE"
print(paste("IS.LOGICAL:", is.logical(x)))
## [1] "IS.LOGICAL: TRUE"
print(paste("IS.COMPLEX:", is.complex(x)))
## [1] "IS.COMPLEX: FALSE"
print(paste("IS.INTEGER:", is.integer(x)))
## [1] "IS.INTEGER: FALSE"
print(paste("IS.RAW:", is.raw(x)))
## [1] "IS.RAW: FALSE"
print(paste("IS.CHARACTER:", is.character(x)))
## [1] "IS.CHARACTER: FALSE"
```

Since **NA** is a special case, there are some specific functions for it.

- `is.na()` - Returns TRUE or FALSE if the value is NA
- `anyNA()` - Returns TRUE or FALSE if any value is NA

```
y = NA
print(paste("CLASS:", class(y)))
```

```
## [1] "CLASS: logical"
```

```
print(paste("MODE:", mode(y)))
```

```
## [1] "MODE: logical"
```

```
print(paste("TYPEOF:", typeof(y)))
```

```
## [1] "TYPEOF: logical"
```

```r
print(paste("IS.NA:", is.na(y)))
```

```
## [1] "IS.NA: TRUE"
```

```r
print(paste("anyNA:", anyNA(y)))
```

```
## [1] "anyNA: TRUE"
```

```r
print(paste("IS.NUMERIC:", is.numeric(y)))
```

```
## [1] "IS.NUMERIC: FALSE"
```

```r
print(paste("IS.LOGICAL:", is.logical(y)))
```

```
## [1] "IS.LOGICAL: TRUE"
```

```r
print(paste("IS.COMPLEX:", is.complex(y)))
```

```
## [1] "IS.COMPLEX: FALSE"
```

```r
print(paste("IS.INTEGER:", is.integer(y)))
```

```
## [1] "IS.INTEGER: FALSE"
```

```r
print(paste("IS.RAW:", is.raw(y)))
```

```
## [1] "IS.RAW: FALSE"
```

```r
print(paste("IS.CHARACTER:", is.character(y)))
```

```
## [1] "IS.CHARACTER: FALSE"
```

Comparison Operators

Operator	Name	Example
==	Equal	x == y
!=	Not Equal	x != y
>	Greater Than	x > y
<	Less Than	x < y
>=	Greater Than or Equal To	x >= y
<=	Less Than or Equal To	x < = y

```r
x = 10
y = 11

sprintf("Is X Equal to Y: %s", x==y)
```

```
## [1] "Is X Equal to Y: FALSE"
```

```r
sprintf("Is X Not Equal to Y: %s", x!=y)
```

```
## [1] "Is X Not Equal to Y: TRUE"
```

```r
sprintf("Is X Greater Than Y: %s", x>y)
```

```
## [1] "Is X Greater Than Y: FALSE"
```

```r
sprintf("Is X Less Than Y: %s", x<y)
```

```
## [1] "Is X Less Than Y: TRUE"
```

```r
sprintf("Is X Greater Than or Equal to Y: %s", x>=y)
```

```
## [1] "Is X Greater Than or Equal to Y: FALSE"
```

```r
sprintf("Is X Less Than or Equal to Y: %s", x<=y)
```

```
## [1] "Is X Less Than or Equal to Y: TRUE"
```

Operator	Name	D escription	Example
&	E l ement-wise AND operator	Returns True if both elements are TRUE	`c( T R UE , F ALS E , TR UE) & c (F AL S E , T RUE,TRUE)` Returns: `FA LS E , F A LSE, TRUE`
&&	AND operator	Returns TRUE if both statements are TRUE	TRUE && TRUE Returns TRUE TRUE && FALSE Returns FALSE FALSE && TRUE Returns FALSE
|	E l ement-wise OR operator	Returns TRUE if one or more element is TRUE	`c ( TRUE , F ALS E , T RUE ) & c (FA L SE , T R UE, TRUE)` Returns: `TR UE , T RUE, TRUE`

Operator	Name	D escription	Example
||	OR operator	Returns TRUE if at least one statement is TRUE	TRUE && TRUE Returns TRUE TRUE && FALSE Returns TRUE FALSE && TRUE Returns TRUE
!	Logical NOT	Negates any statement: Returns TRUE if statement is FALSE and vice versa	! FALSE Returns TRUE ! TRUE Returns FALSE

D. Numeric

Decimal values are called numeric in R. It is the default R datatype for numbers in R.

Real numbers with a decimal point are represented using this data type in R. It uses a format for double-precision floating-point numbers to represent numerical values.

Any value 1, 2, 2.544, 1E123 is evaluated as a Numeric datatype

```
1
## [1] 1
3
## [1] 3
2.53
## [1] 2.53
2.23E-2
## [1] 0.0223
```

Remember Numerics are essentially floating point numbers. Meaning, that there is a limited precision the numbers can store and when comparing numbers, we need to beware of imprecision.

```
x = 3.223423234E-10 * 10.0
print(x)
```

```
## [1] 3.223423e-09
```

```
# Comparison
print(x==3.223423234E-9)
```

```
## [1] FALSE
```

In R, a Numeric can be also be (*NaN*, *Inf*, *-Inf*).

- **NaN** - Not a Number
- **Inf** - Infinite (positive)
- **-Inf** - Infinite (negative)

```
x <- 1.0
print(paste("CLASS:", class(x)))
## [1] "CLASS: numeric"
print(paste("MODE:", mode(x)))
## [1] "MODE: numeric"
print(paste("TYPEOF:", typeof(x)))
## [1] "TYPEOF: double"
print(paste("IS.NUMERIC:", is.numeric(x)))
## [1] "IS.NUMERIC: TRUE"
print(paste("IS.LOGICAL:", is.logical(x)))
## [1] "IS.LOGICAL: FALSE"
print(paste("IS.COMPLEX:", is.complex(x)))
## [1] "IS.COMPLEX: FALSE"
print(paste("IS.INTEGER:", is.integer(x)))
## [1] "IS.INTEGER: FALSE"
print(paste("IS.RAW:", is.raw(x)))
## [1] "IS.RAW: FALSE"
print(paste("IS.CHARACTER:", is.character(x)))
## [1] "IS.CHARACTER: FALSE"
```

Because there are 3 special cases for numeric, there are also functions to handle these.

- is.finite() - Return True if value is not NaN, Inf, -Inf, or NA
- is.infinite() - Returns True if value is Inf or -Inf
- is.nan() - Returns True if value is NaN

Finite Numbers

```
x = 1.0
print(is.infinite(x))
```

```
## [1] FALSE
```

```
print(is.finite(x))
```

```
## [1] TRUE
```

Infinite Numbers

```
x = Inf
y = -Inf
print(is.infinite(x))
```

```
## [1] TRUE
```

```r
print(is.infinite(y))
```

```
## [1] TRUE
```

```r
print(is.nan(x))
```

```
## [1] FALSE
```

```r
print(is.nan(y))
```

```
## [1] FALSE
```

Not a Number

```r
x = NaN
print(is.nan(x))
```

```
## [1] TRUE
```

E. Integer

R supports integer data types which are the set of all integers. If you want an integer, you'll need to need to add the `L` suffix

```r
10L
```

```
## [1] 10
```

```r
x <-2L
print(paste("CLASS:", class(x)))
## [1] "CLASS: integer"
print(paste("MODE:", mode(x)))
## [1] "MODE: numeric"
print(paste("TYPEOF:", typeof(x)))
## [1] "TYPEOF: integer"
print(paste("IS.NUMERIC:", is.numeric(x)))
## [1] "IS.NUMERIC: TRUE"
print(paste("IS.LOGICAL:", is.logical(x)))
## [1] "IS.LOGICAL: FALSE"
print(paste("IS.COMPLEX:", is.complex(x)))
## [1] "IS.COMPLEX: FALSE"
print(paste("IS.INTEGER:", is.integer(x)))
## [1] "IS.INTEGER: TRUE"
print(paste("IS.RAW:", is.raw(x)))
## [1] "IS.RAW: FALSE"
print(paste("IS.CHARACTER:", is.character(x)))
## [1] "IS.CHARACTER: FALSE"
```

Numerical Datatype Arithmetic R provides a set of standard arithmetic operators to do basic math.

Basic Arithmetic Operators

R Operator	Name
+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Exponent/Power
%%	Modulus
%/%	Integer Division
sqrt()	Square root
abs()	Absolute Value

```
x <- 124
print(paste("Addition:", x+5))
## [1] "Addition: 129"
print(paste("Subtraction", x-324))
## [1] "Subtraction -200"
print(paste("Multiplication:", x*0.2421))
## [1] "Multiplication: 30.0204"
print(paste("Division:", x/21))
## [1] "Division: 5.90476190476191"
print(paste("Exponent/Power:", x^0.5))
## [1] "Exponent/Power: 11.13552872566"
print(paste("Modulus:", x%%21))
## [1] "Modulus: 19"
print(paste("Integer Division:", x%/%21))
## [1] "Integer Division: 5"

print(paste("Grouping:", 2*(x+12)))
## [1] "Grouping: 272"
print(paste("Different Grouping:", (2*x)+12))
## [1] "Different Grouping: 260"
print(paste("Without Grouping:", 2*x+12))
## [1] "Without Grouping: 260"
```

Grouping for Precedence If you need to group operations for precedence, you can use parentheses `(` and `)`.

```
print(paste("Grouping:", 2*(x+12)))
## [1] "Grouping: 272"
print(paste("Different Grouping:", (2*x)+12))
## [1] "Different Grouping: 260"
print(paste("Without Grouping:", 2*x+12))
## [1] "Without Grouping: 260"
```

F. Character

R supports character data types where you have all the alphabets and special characters. It stores character values or strings. Strings in R can contain alphabets, numbers, and symbols.
The easiest way to denote that a value is of character type in R data type is to wrap the value inside single or double quotes.

'string"

"sdd'

```
'string'
## [1] "string"
"String"
## [1] "String"

'string"
"sdd'
## [1] "string\"\n\"sdd"
```

```
x = "A"
print(paste("CLASS:", class(x)))
## [1] "CLASS: character"
print(paste("MODE:", mode(x)))
## [1] "MODE: character"
print(paste("TYPEOF:", typeof(x)))
## [1] "TYPEOF: character"
print(paste("IS.NUMERIC:", is.numeric(x)))
## [1] "IS.NUMERIC: FALSE"
print(paste("IS.LOGICAL:", is.logical(x)))
## [1] "IS.LOGICAL: FALSE"
print(paste("IS.COMPLEX:", is.complex(x)))
## [1] "IS.COMPLEX: FALSE"
print(paste("IS.INTEGER:", is.integer(x)))
## [1] "IS.INTEGER: FALSE"
print(paste("IS.RAW:", is.raw(x)))
## [1] "IS.RAW: FALSE"
print(paste("IS.CHARACTER:", is.character(x)))
## [1] "IS.CHARACTER: TRUE"
```

II. Casting or Changing Datatypes

Sometimes, you need to convert from one Datatype to another. The process of converting datatypes is also called **casting**

R has a an `as.<type>` function that will attempt to cast/change a variable from it's current datatype to the `type` specified.

Function	Description
as.numeric()	Coerce data to `numeric`
as.logical()	Coerce data to `logical`
as.complex()	Coerce data to `complex`
as.integer()	Coerce data to `integer`

Function	Description
as.raw()	Coerce data to `raw`
as.character()	Coerce data to `character`

```r
x <- Inf
y = as.logical(x)
print(paste("AS LOGICAL:", y))
## [1] "AS LOGICAL: TRUE"

y = as.complex(x)
print(paste("AS COMPLEX:", y))
## [1] "AS COMPLEX: Inf+0i"

y = as.integer(x)
## Warning: NAs introduced by coercion to integer range
print(paste("AS INTEGER:", y))
## [1] "AS INTEGER: NA"

y = as.raw(x)
## Warning: NAs introduced by coercion to integer range
## Warning: out-of-range values treated as 0 in coercion to raw
print(paste("AS RAW:", y))
## [1] "AS RAW: 00"

y = as.character(x)
print(paste("AS CHARACTER:", y))
## [1] "AS CHARACTER: Inf"
```

Casting Numerical

```r
x <- TRUE
y = as.numeric(x)
print(paste("AS LOGICAL:", y))
## [1] "AS LOGICAL: 1"

y = as.complex(x)
print(paste("AS COMPLEX:", y))
## [1] "AS COMPLEX: 1+0i"

y = as.integer(x)
print(paste("AS INTEGER:", y))
## [1] "AS INTEGER: 1"

y = as.raw(x)
print(paste("AS RAW:", y))
## [1] "AS RAW: 01"
```

```r
y = as.character(x)
print(paste("AS CHARACTER:", y))
## [1] "AS CHARACTER: TRUE"
```

Casting Boolean

```r
x <- "Hellow"
y = as.logical(x)
print(paste("AS LOGICAL:", y))
## [1] "AS LOGICAL: NA"

y = as.complex(x)
## Warning: NAs introduced by coercion
print(paste("AS COMPLEX:", y))
## [1] "AS COMPLEX: NA"

y = as.integer(x)
## Warning: NAs introduced by coercion
print(paste("AS INTEGER:", y))
## [1] "AS INTEGER: NA"

y = as.raw(x)
## Warning: NAs introduced by coercion
## Warning: out-of-range values treated as 0 in coercion to raw
print(paste("AS RAW:", y))
## [1] "AS RAW: 00"

y = as.numeric(x)
## Warning: NAs introduced by coercion
print(paste("AS NUMERIC:", y))
## [1] "AS NUMERIC: NA"
```

Casting Character

```r
x <- "1.25"
y = as.logical(x)
print(paste("AS LOGICAL:", y))
## [1] "AS LOGICAL: NA"

y = as.complex(x)
print(paste("AS COMPLEX:", y))
## [1] "AS COMPLEX: 1.25+0i"

y = as.integer(x)
print(paste("AS INTEGER:", y))
## [1] "AS INTEGER: 1"
```

```r
y = as.raw(x)
print(paste("AS RAW:", y))
## [1] "AS RAW: 01"

y = as.numeric(x)
print(paste("AS NUMERIC:", y))
## [1] "AS NUMERIC: 1.25"
```

Casting Numerical Strings

III. More Indepth Look at Characters and Strings

R is 1-based

Well, R is one of the few languages that is **1-based**. So, the first position in a string is 1 not 0 like in many other programming languages. This is an important point to remember, because it's a fundamental difference. When we learn about vectors, lists, and other datatypes - you'll need to remember that **R is 1-based** not 0-based.

Character String Functions

Some of the most useful character/string function in R.

R Function	Description
print(x)	Print its arguments
paste(x,x)	Concatenate String
sprintf(x)	C-style format a string
substr(x, start, stop)	Get Substring
nchar(x)	Get String Length
sub(pattern, replacment, x)	Replace substring with
tolower(x)	Convert string to lowercase
toupper(x)	Convert string to uppercase
trimws(x)	Strip whitespace characters
strrep(x)	Repeat string n times
strsplit(x)	Split up string
chartr(x)	Translate characters in characters string

Here's some example code to show the basic built-in functions.

```r
x = " Hello World! How are you? "
print(x)
## [1] " Hello World! How are you? "
```

Paste

```r
print(paste("Pasting:", 1.45, "and other text"))  # Concatenating a number
```

```
## [1] "Pasting: 1.45 and other text"
```

Substring

```
y = substr(x,1,10)
print(paste("Substring:", y))
```

```
## [1] "Substring:  Hello Wor"
```

Number of Characters (nchar)

```
print(paste("x is", nchar(x), "characters long."))
```

```
## [1] "x is 27 characters long."
```

Uppercase and Lowercase

```
print(paste("Lowercase:", tolower(x)))
```

```
## [1] "Lowercase:  hello world! how are you? "
```

```
print(paste("Uppercase:", toupper(x)))
```

```
## [1] "Uppercase:  HELLO WORLD! HOW ARE YOU? "
```

Trim Whitespace

```
print(paste("Trim Whitespace:", trimws(x), sep=""))
```

```
## [1] "Trim Whitespace:Hello World! How are you?"
```

Repeat String

```
print(paste("Repeated String:", strrep('ABC,', 10)))
```

```
## [1] "Repeated String: ABC,ABC,ABC,ABC,ABC,ABC,ABC,ABC,ABC,ABC,"
```

Split String

```
print(paste("Split String:", strsplit(x, " "))) #Notice its outputting a Vector here
```

```
## [1] "Split String: c(\"\", \"Hello\", \"World!\", \"How\", \"are\", \"you?\")"
```

```
y =  strsplit(x, " ")

print(paste("Class:", class(y), ",Mode:", mode(y), ",Typeof:", typeof(y)))
```

```
## [1] "Class: list ,Mode: list ,Typeof: list"
```

```r
print(y)
```

```
## [[1]]
## [1] ""       "Hello"  "World!" "How"    "are"    "you?"
```

Notice that we get a *Class:*`list`. We'll discuss what a `list` is upcoming sections.

Translate Characters

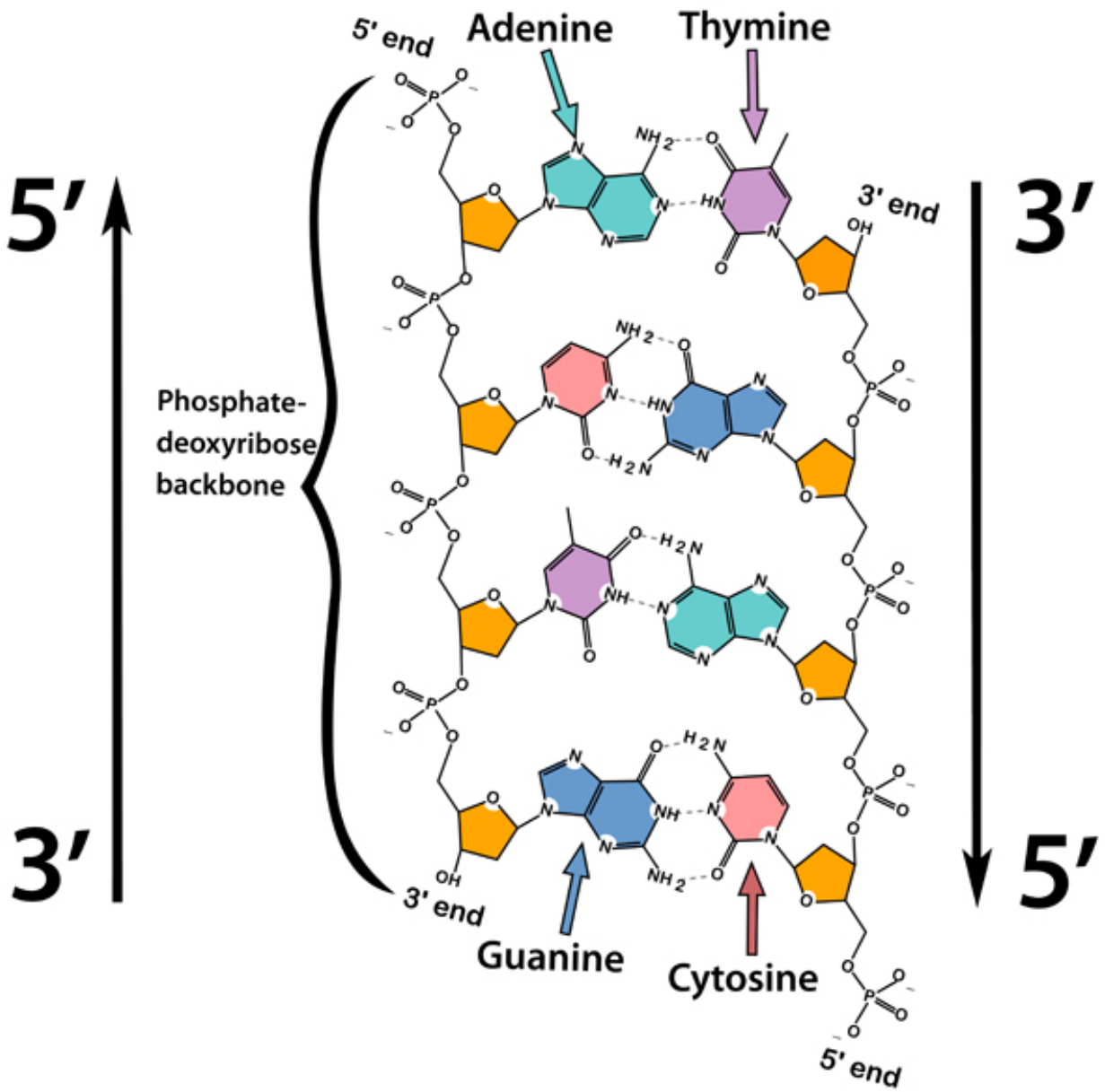
In this example, we'll change all the lower-case vowels to uppercase.

```r
print(chartr("aeiou", "AEIOU",x))
```

```
## [1] " HEllO WOrld! HOw ArE yOU? "
```

Getting Reverse Complement of DNA in R

DNA is has two complementary strands: one runs from the 5' -> 3' and the other runs 3' ->5'.



The first completed sequence of the whole Human genome was completed in 2004. It was composed of 25 contigs (continuous stretches of DNA. ie Chromosomes: Autosomes 1 to 22, the Mitochondria DNA, and Sex chromosomes (X,Y). To save space, a choice was made to only distribute the 5' to 3' strand in a computer format called FASTA. The data from the 3' to 5' wasn't lost - because it was just the complement of the 5' to 3' strand in reverse.

If a gene's coding sequence starts on 5' to 3' strand with an **ATG** then it's easy to find and you can pass it to function that parse the coding regions. If you didn't take the reverse complement of the sequence it'd mean you had to have a separate programs and models to detect genes on the reverse strand. For example you'd to rewrite your complete programs to look for a CAT and then work in the backward.

It's much easier and quicker to just take the reverse complement of the DNA/RNA sequence. To reverse complement DNA/RNA, we need to:

1. **Change the characters (ie Complement the DNA/RNA Sequence)**

- DNA: ACGT -> TGCA
- RNA: ACGU -> UGCA

2. **Reverse the order of the characters in the string**

1. Complement the Sequence To complement the sequence, we can do it all at the same time using `chartr`

```
dna <- 'ACGTTTAC'
complement = chartr("ACGT", "TGCA", dna)
print(complement)
```

```
## [1] "TGCAAATG"
```

2. Reverse the Sequence Without `stringi` package, we'd need to split the string into it's individual characters and then reverse the vector of character and then combine them together again.

```
dna_split = strsplit(complement, "")[[1]]  # Split up into vector of individual characters
reversed = rev(dna_split) # Reverse the order of the vector
rev_comp = paste(reversed, collapse="")  # Concatenate the characters with no spaces
print(rev_comp)
```

```
## [1] "GTAAACGT"
```

Extending Base String Functionality (Package: stringi and/or stringr)

However, using `stringi` library, we can perform the operation in a single command

```
library(stringi)
rev_comp = stri_reverse(complement)
print(rev_comp)
```

```
## [1] "GTAAACGT"
```

Stringi Vignette

Importing FASTA File in R using seqinr

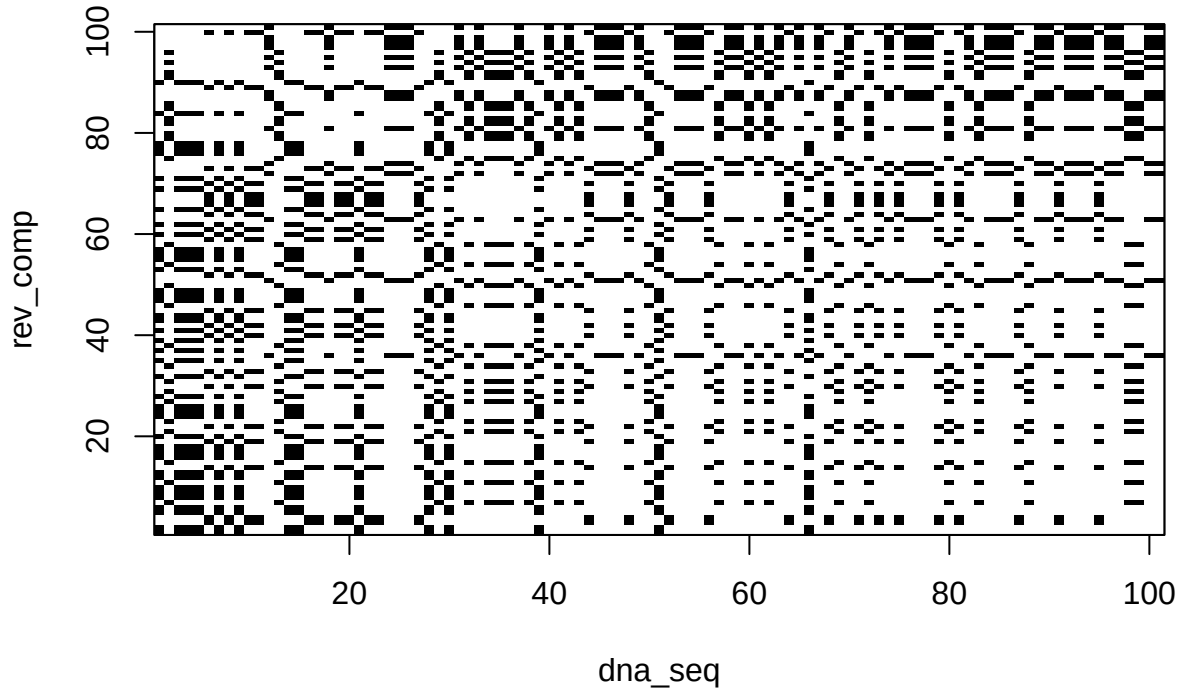
SeqinR is a small library in R that allows you to easily import a FASTA file sequence file.

```
if (!requireNamespace("seqinr", quietly = TRUE)) {
  install.packages("seqinr", repos = "http://cran.us.r-project.org")
}
```

Importing FASTA File

```
>Example Fasta
NNNN
```

```r
dotPlot(dna_seq, rev_comp)
```



Stringr

In addition to working with Unicode and International encoded text, `stringr` also provides equivalent functions to the R-base character functions and provides many additional functions.

Let's take a look a `stringr` library.

- Stringr Cheatsheet

Basic String Function

Functions	Description
str_length(x)	Returns number of characters Equivalent to nchar()

Detecting Matches

The patterns below can be a simple text match or more complex regular expressions.

Remember, we briefly discussed Regular Expressions in Unix, which are a way to represent different text patterns. (The second page of Stringr Cheatsheet is dedicated to Regular Expressions.) A simple example of a regular expression for DNA might look something like this `[ACGTNacgtn]+` , which would match any string with any composition of A, C, G, T or N of one or more character length.

Functions	Description
str_detect(x, pattern, negate=FALSE)	Detect the presence of a pattern. Returns Boolean
str_starts(x, pattern, negate=FALSE)	Detect the presence of a pattern at the beginning of string
str_ends(x, pattern, negate=FALSE)	Detect the presence of a pattern at the ending of string
str_which(x, pattern, negate=FALSE)	Find the indexes of strings that contain a pattern match
str_locate(x, pattern)	Locate the first position of pattern matches in a string
str_locate_all(x, pattern)	Locate the all positions of pattern matches in a string
str_count(x, pattern)	Count the number of matches in a string

```r
#install.packages("stringr")
library(stringr)
dna = "ttcccagcacagaggaaaaggtagatctgaatgctgaacccctatatggaagaagaaaac
tgaacaaacagaaattgtcatgctctgagagccctgaggatccccaagagatgacttgga
tgacttcgaagagtagcctacagaaagttaatgattggttttctagaagtgatgatgtat
taacttctgatgatttccatgacgaagggtctaattcaaatacaaaagctgaggcggaag
aaatcccaagtgcagcagatggggtttttgtttcttcagagannnnnnnnnnnnnnnnn
nnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnngtagaga
gtagcattgaagataaaatatttgggaaaacttatcggaggaaagcaagcttcgctaact
tgaactgcacaactgaagatgtaactctagaatcatctctactagaaccgcatatggcac
acaaacacccttcacaaataaattaaaacgtaaaagaattacatcaagccttggtcctg
aggattttataaagaaagtagatttggcggttgttgttcaaaagtctcctgaaaagaaaa
tcgagaggctcaaccaaatggatcaaaatggtcaggtggtgaatactacta"

dna
```

```
## [1] "ttcccagcacagaggaaaaggtagatctgaatgctgaacccctatatggaagaagaaaac\ntgaacaaacagaaattgtcatgctctgagagcc
```

```r
print(str_detect(dna, 'ttcc'))
```

```
## [1] TRUE
```

```r
print(str_detect(dna, 'n'))
```

```
## [1] TRUE
```

```r
print(str_starts(dna, 'ttcc'))
```

```
## [1] TRUE
```

```r
print(str_ends(dna, 'acta'))
```

```
## [1] TRUE
```

```r
print(str_which(dna,'ggg'))
```

```
## [1] 1
```

```r
print(str_locate(dna,'ggg'))
```

```
##      start end
## [1,]   210 212
```

```r
print(str_locate_all(dna,'ggg'))
```

```
## [[1]]
##      start end
## [1,]   210 212
## [2,]   265 267
## [3,]   390 392
```

```r
print(str_count(dna, 'a'))
```

```
## [1] 209
```

```r
print(str_count(dna, 'g'))
```

```
## [1] 129
```

Subsetting & Extracting Matching Strings

Functions	Description
str_sub(x, start, end)	Extract substring
str_subset(x, pattern, negate=FALSE)	Return string that contains a match
str_extract(x, pattern)	Return first match found in each string
str_extract_all(x, pattern)	Return every match found in each string
str_match(x, pattern)	Locate the first match found in each string as matrix
str_match_all(x, pattern)	Locate the all matches found in each string as matrix

```r
print(paste("Substring:", str_sub(dna, 0,100)))
```

```
## [1] "Substring: ttcccagcacagaggaaaaggtagatctgaatgctgaacccctatatggaagaagaaaac\ntgaacaaacagaaattgtcatg
```

```r
print(paste("Subset:", str_subset(dna, 'ttc')))
```

```
## [1] "Subset: ttcccagcacagaggaaaaggtagatctgaatgctgaacccctatatggaagaagaaaac\ntgaacaaacagaaattgtcatgctc
```

```r
print(paste("Extract:", str_extract(dna, 'ttcg')))
```

```
## [1] "Extract: ttcg"
```

```r
print(paste("Extract All:", str_extract_all(dna, 'ttcg')))
```

```
## [1] "Extract All: c(\"ttcg\", \"ttcg\")"
```

```r
print(paste("String Match:", str_match(dna, 'ttcg')))
```

```
## [1] "String Match: ttcg"
```

```r
print(paste("All String Matches:", str_match_all(dna, 'ttcg')))
```

```
## [1] "All String Matches: c(\"ttcg\", \"ttcg\")"
```

Mutate Strings

Functions	Description
str_sub()<-value	Identify with str_sub then replace
str_replace(string, pattern, replacement)	Replace the first match in string
str_replace_all(string, pattern, replacement)	Replace all matches in string
str_remove(string, pattern)	Remove first match in string
str_remove_all(string, pattern)	Remove all matches in string
str_to_lower(string)	Convert string to uppercase
str_to_upper(string)	Convert string to lowercase
str_to_title(string)	Convert string to camelcase
str_trim(string)	Remove leading and ending whitespace

```r
x = "Hello World"
str_sub(x,1,5)<-'goodbye'
print(paste("str_sub:", x))
```

```
## [1] "str_sub: goodbye World"
```

```r
print(paste("str_replace:", str_replace(x,'o','e')))
```

```
## [1] "str_replace: geodbye World"
```

```r
print(paste("str_replace_all", str_replace_all(x,'o','e')))
```

```
## [1] "str_replace_all geedbye Werld"
```

```r
print(paste("str_remove:", str_remove(x,'o')))
```

```
## [1] "str_remove: godbye World"
```

```r
print(paste("str_remove_all:", str_remove_all(x,'o')))
```

```
## [1] "str_remove_all: gdbye Wrld"
```

```r
print(paste("str_to_upper:", str_to_upper(x)))
```

```
## [1] "str_to_upper: GOODBYE WORLD"
```

```r
print(paste("str_to_lower:", str_to_lower(x)))
```

```
## [1] "str_to_lower: goodbye world"
```

```r
print(paste("str_to_title:", str_to_title(x)))
```

```
## [1] "str_to_title: Goodbye World"
```

```r
text = '\nHello\nWorld\r\n'
print(text)
```

```
## [1] "\nHello\nWorld\r\n"
```

```r
print(str_trim(text))
```

```
## [1] "Hello\nWorld"
```

Join and Split Strings

Functions	Description
str_c(..., sep=" ", collapse=NULL)	Combines multiple strings to a single string (like paste0)
str_flatten(string, pattern, replacement)	Combined multiple strings to a single string
str_dup(string,n)	Repeat string n times
str_unique(string)	Remove duplicate strings
str_split_fixed(string, pattern)	Split vector of strings into matrix of strings
str_split(string)	Return a list of substrings
str_glue(string)	Creates a string from string and expressions

```r
print(str_c("Letter: ", letters))
```

```
##  [1] "Letter: a" "Letter: b" "Letter: c" "Letter: d" "Letter: e" "Letter: f"
##  [7] "Letter: g" "Letter: h" "Letter: i" "Letter: j" "Letter: k" "Letter: l"
## [13] "Letter: m" "Letter: n" "Letter: o" "Letter: p" "Letter: q" "Letter: r"
## [19] "Letter: s" "Letter: t" "Letter: u" "Letter: v" "Letter: w" "Letter: x"
## [25] "Letter: y" "Letter: z"
```

```r
print(str_flatten(letters, "-"))
```

```
## [1] "a-b-c-d-e-f-g-h-i-j-k-l-m-n-o-p-q-r-s-t-u-v-w-x-y-z"
```

```r
print(str_dup("apple", 2))
```

```
## [1] "appleapple"
```

```r
print(str_unique(c("a", "b", "c", "b", "a")))
```

```
## [1] "a" "b" "c"
```

```r
print(str_split_fixed("apple", "", 4))
```

```
##      [,1] [,2] [,3] [,4]
## [1,] "a"  "p"  "p"  "le"
```

```r
print(str_split("apple", ""))
```

```
## [[1]]
## [1] "a" "p" "p" "l" "e"
```

```r
name = "Paul"
print(str_glue("My name is {name}, not {{name}}."))
```

```
## My name is Paul, not {name}.
```

The actual percentage of G and C in sequence can greatly influence the sequence stability, gene expression, and genome function. For this next assignment, you'll be required to calculate the GC percentage of a DNA sequence.

Assignment 1: Calculate the GC Percentage of DNA Sequence

The percentage of Guanine and Cytosine in DNA or RNA significantly impacts it's molecular stability, shape, and function.

$$GCpercent = (\sum G + C)/\sum A + C + G + T$$

Using the R string functions (I'd encourage you to use the `stringr` library), calculate the GC percentage for the following DNA sequence. Be certain to exclude all ambiguous nucleotides or whitespace characters.

```r
library(stringr)
dna = "ttcccagcacagaggaaaaggtagatctgaatgctgaacccctatatggaagaagaaaac
tgaacaaacagaaattgtcatgctctgagagccctgaggatccccaagagatgacttgga
tgacttcgaagagtagcctacagaaagttaatgattggttttctagaagtgatgatgtat
taacttctgatgatttccatgacgaagggtctaattcaaatacaaaagctgaggcggaag
aaatcccaagtgcagcagatggggtttttgtttcttcagagagtagagannnnnnnnnn
nnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnn
gtagcattgaagataaaatatttgggaaaacttatcggaggaaagcaagcttcgctaact
tgaactgcacaactgaagatgtaactctagaatcatctctactagaaccgcatatggcac
acaaacacccttcacaaataaattaaaacgtaaaagaattacatcaagccttggtcctg
aggattttataaagaaagtagatttggcggttgttgttcaaaagtctcctgaaaagaaaa
tcgagaggctcaaccaaatggatcaaaatggtcaggtggtgaatactacta"

print(dna)
```

```
## [1] "ttcccagcacagaggaaaaggtagatctgaatgctgaacccctatatggaagaagaaaac\ntgaacaaacagaaattgtcatgctctgagagcc
```

Get the GC Percentage of Chromosome 20 with all N (ambiguous characters) removed.

```r
fasta_string = getSequence(my_fasta[[1]], as.string=TRUE)  #Get the sequence
```